



PIIPS

Precision Intelligent Invoice Processing Suite · v2.2

Architecture, schema, and internals — for whoever maintains this application next.

Architecture

A FastAPI backend, a Vite/React single-page frontend, and a SQL Server database, fronted by IIS as a reverse proxy.

```

Browser (React SPA, frontend/dist)
  | HTTPS
  v
IIS site "PIIPS_AI" --- web.config: URL Rewrite -> http://127.0.0.1:8000
  |
  v
Uvicorn (NSSM service "PIIPS_Backend", 127.0.0.1:8000)
  |
  v
FastAPI app (app.py) ---calls---> Service First / NAV API (config: sf_api_url)
  |
  v
SQL Server (database.py, pyodbc) | Working folder (config_store: folder_path)
                                | Input / New_Format / Unsupported /
                                | Incomplete Data / ALL_INVOICES / output

```

TERM	MEANING
Purchase Header/Line/Reservation	The three sheets an extracted invoice becomes: one header row, N line rows, N reservation rows.
Template	A per-vendor extraction profile: entity, invoice type, PO number format, static values.
Tracker	tbl_Purchase_Tracker — one row per invoice, carrying its lifecycle status and audit trail.
Lifecycle	LOADED → POSTED → COMPLETE, driven by the Load/Post/Complete screens.

Backend module map

FILE	RESPONSIBILITY
app.py	FastAPI routes: auth, users, mail settings, announcements, config, invoices, publish, manuals.
database.py	All SQL Server access — schema DDL (idempotent, run on every startup/DB-config-save), stored procedures, and the Python wrappers around them.
processor.py / anchor_extract.py / ocr_engine.py / invoice_schema.py	PDF text extraction, template/anchor matching, field parsing.
mailer.py	SMTP sending (stdlib smtplib only) + the four HTML email templates.
config_store.py	config.json read/write — folder_path, db_connection, sf_api_url, publish_root.
secret_store.py	Windows DPAPI encrypt/decrypt (LOCAL_MACHINE scope) for secrets at rest.
deploy.py	The Super Admin Publish feature — git pull/merge, frontend build, robocopy to a local staging folder.

v2.2 note. Column detection in anchor_extract.py was made more robust: the three-column Bill To / Ship To / Invoice Details header is now located structurally even when a vendor's Invoice Details column carries no literal caption, and OCR word-continuation handling in ocr_engine.py was fixed so wrapped or overlapping word boxes stitch back together correctly instead of dropping or misordering text. See CHANGELOG.md for the full list of extraction fixes in this release.

Database schema (this release)

Schema changes are idempotent DDL run automatically — on every app startup, and again whenever Database Configuration is saved (so pointing at a brand-new empty database just works).

TABLE	NOTES
tbl_user	+ <code>Email</code> , <code>MustChangePassword</code> columns this release.
tbl_MailSettings	New. SettingID IDENTITY(31,1); single row; Password is DPAPI-encrypted.
tbl_Announcement	New. Title, BodyText, ImagePath, VideoUrl, EndDateTime, IsActive, Stopped* audit columns.
tbl_Purchase_Header / Line / Tracker, tbl_Reservation_Entry, tbl_InputFile_Log	Unchanged this release.

PIIPS_LIVE bootstrap. `PIIPS.sql` (git-ignored, local artifact only — it contains a real password hash) drops and recreates a target database from scratch: full current schema plus master data only (user types, statuses, a clean default Sadmin, blank mail-settings password, business templates). Regenerate it from a live database's actual schema rather than hand-editing it if the schema has moved on.

Authentication & password policy

Every password on PIIPS is system-generated (`database.generate_temp_password()`) and emailed — there is no code path where an admin types a password for someone else. Policy (`validate_password_policy`): minimum 5 characters, at least one uppercase, one lowercase, one digit, one of `!@#$$%^&*()-_+=[]{};:,.<>?/`. Hashing is PBKDF2-SHA256 (`hash_password` / `verify_password`).

Who can reset whose password

Enforced server-side in `POST /api/users/reset-password` (mirrored client-side by `UserManagement.jsx`'s `canAssignFor`): Super Admin → anyone including themselves; Admin → only plain User/Accounts targets, never themselves, another Admin, or a Super Admin.

The default Sadmin account

`DEFAULT_SUPER_ADMIN_USERNAME = "Sadmin"`, seeded by `ensure_default_super_admin()` on every startup if missing — fixed password, no email, never forced to change it, so it can never be locked out by mail-server misconfiguration. Any number of other Super Admins may also exist, created normally with email + a generated password. Both `POST /api/users/active` (deactivating) and the User Management UI explicitly block anyone from deactivating Sadmin, and block deactivating your own account — added this release after a real self-lockout during testing (a Super Admin's own row had no such guard).

Email (mailer.py)

stdlib `smtplib` / `email.mime` only — no new dependency. SMTP account lives in `tbl_MailSettings`, edited via the Mail Server Setting screen (Super Admin only), saved only after a live connect+login test succeeds.

Four templates, one shell

`welcome_email_html`, `password_reset_email_html`, `deactivation_email_html`, `activation_email_html` — all share `_shell()`: an indigo-to-magenta gradient header matching the app's logo, a monospace password chip, and the PIIPS mark embedded as a base64 `data:` URI (not a URL reference — a `request.base_url` - derived image URL is only reachable from wherever the request happened to originate, not necessarily from the actual recipient's device/network).

Sign-in links point at the right environment

`_base_url(request)` (app.py) prefers the caller's browser `Origin` / `Referer` header over `request.base_url` — the latter reflects Uvicorn's own view of itself behind the IIS reverse proxy (`http://127.0.0.1:8000`), which is wrong on every environment unless IIS forwards proxy headers Uvicorn is configured to trust (it isn't here). Fixed this release after Live-environment emails were linking back to the internal address.

?signout=1

Every "Sign in to PIIPS" link carries this param. `App.jsx`'s `loadUser()` checks for it, clears any already-active session, and strips it from the URL — a welcome/reset email is often opened in the same browser an admin just used to create the account.

Announcements

Super Admin publishes a site-wide notice (`tbl_Announcement`): text, an optional image (saved under `announcement_media/`, git-ignored and excluded from Publish), an optional video URL, and an end date/time; stoppable early at any point.

`AnnouncementBell` (`components.jsx`) polls `GET /api/announcements/active` every 15 seconds and also listens for a same-tab `ANNOUNCEMENT_CHANGED_EVENT` DOM event (dispatched right after a publish/stop) so the publisher's own session updates instantly rather than waiting on the next poll. Read state is tracked client-side in `localStorage` (`piips_read_announcements`) — deliberately a bell + popup, not an inline banner, so it never disturbs whatever the user is doing on the Dashboard.

Configuration & secrets

SETTING	WHERE
folder_path, db_connection, sf_api_url, publish_root	config.json (git-ignored, machine-specific)
Mail SMTP account	tbl_MailSettings (DB), Password DPAPI-encrypted

`secret_store.protect()` / `unprotect()` wrap Windows DPAPI, LOCAL_MACHINE scope, marked with an `enc:dpapi:` prefix. `config_store.ensure_secret_encrypted()` runs on every startup and re-encrypts a plaintext `db_connection` in place.

Machine-locked, every time. A DPAPI-encrypted value copied from one machine (config.json's db_connection, or tbl_MailSettings.Password) is **useless** on a different machine — it can only be decrypted where it was encrypted. Always re-enter it through the app's own UI after moving to a new environment; never copy the raw encrypted value across machines.

Deployment (summary)

See **PIIPS_UAT_Deployment_Guide.pdf** (this manual's companion document, Super Admin only) for the full step-by-step: IIS/ARR reverse-proxy setup, the NSSM service, and the common-problems list. In short:

- **Publish** (Super Admin menu) stages a build locally — git pull/merge the right branch, `npm run build`, robocopy to a staging folder. It does not touch the remote server.
- Moving a staged build onto the real server, and restarting the service there, is still a manual step (robocopy `/MIR` + `Restart-Service PIIPS_Backend`).
- **web.config** must remove the WebDAV module/handler if the server has that IIS role feature installed — otherwise it silently intercepts POST/PUT/DELETE (GET still works), which surfaced this release as "Method Not Allowed" on every save action. Harmless to remove when WebDAV isn't present.

What's New in v2.3 — Technical Notes

Backend/schema/frontend changes from this release, for whoever maintains this next.

Template Active/Inactive

tbl_Template already had a soft-delete (IsActive) - the Template screen now exposes both directions. New/changed pieces:

- `database.activate_template()` - new, mirrors `delete_template()`, calls a new `usp_ActivateTemplate` stored proc that restores the header + every static value that was active at the moment it was deactivated.
- `usp_GetTemplates` no longer filters `IsActive=1` - it returns every template, active and inactive, with the flag included. Any caller that must only ever use an ACTIVE template (live processing) is responsible for checking it themselves now - see `template_store.static_for_path()`, which explicitly rejects an inactive entry.
- `template_store.save_template()` now blocks a rename or a same-name create when the target key already exists (active or inactive), and blocks a rename once `database.template_folder_has_batches()` says the template has any batch behind it.
- `database.template_incomplete_batches()` blocks deactivation until every batch is Completed.

Known gap fixed this release: both guard functions originally matched batches purely via `tbl_InputFile_Log.RelPath` joined to the tracker. That table is only ever written by the File Explorer's own upload endpoint (`usp_LogInputFiles`) - a file placed into Input any other way (a network copy, a script) never gets a row there, so the guard silently saw nothing and let a deactivate/rename through on a real, unfinished batch. Fixed by matching `tbl_Purchase_Tracker.BatchName` directly instead (always populated once Start has run, regardless of how the file arrived - see `_template_batch_name_pattern()`, which reverses `processor.py`'s own `_batch_name_for()` sanitization), plus a direct filesystem listing of the template's Input folder (`_template_pending_input_files()`) for files that haven't been processed at all yet. If you're extending either guard, do not reintroduce a dependency on `tbl_InputFile_Log` alone.

Template History (audit)

Activate/deactivate now call `log_event()` with a new entity value, `Entity='TEMPLATE'` (actions `TEMPLATE_ACTIVATED` / `TEMPLATE_DEACTIVATED`) - same `tbl_Audit_Event` table invoice history already used. The template key is stored in the audit row's `InvoiceNo` column (repurposed, not renamed, to avoid a schema change) so `database.get_template_history(key)` can look it up by exact match. New endpoint: `GET /api/templates/history?key=...` (query param, not a path segment, since a template key contains backslashes).

Invoice Search: Details

New `database.get_invoice_details(header_id)` returns every column `excel_export.sheet_columns()` currently lists for Purchase Header/Line/Reservation Entry (not just the mandatory subset `get_invoice_field_check` uses) with each column's current stored value - no who/when. PART only; raises `ValueError` for SERVICE (caught as HTTP 400). New endpoint: `GET /api/invoices/{header_id}/details`. Frontend: the shared results-table + Details/History/View logic used by both Invoice Search and Completed Invoices was extracted into `InvoiceResultsPanel.jsx` to avoid duplicating it.

New menu: Completed Invoices

New `database.completed_invoices()`: starts from every COMPLETED tracker row, then filters to only the ones whose archive copy actually exists in ALL_INVOICES right now (matching that folder's own naming convention, <Invoice No.>_<Vendor Name><ext>, built the same way `app.py`'s `lifecycle_advance` names the archive copy). The row's `archived_name` field carries that computed name for display; `file_name` stays the ORIGINAL upload name so View/Download keep working off the real file in its status folder.

Known fragility, not yet fixed: the archived filename is recomputed from the invoice's CURRENT `invoice_no/vendor` at query time, not stored at archive time. If either field is edited after the invoice completes (Recheck All, a manual fix), the recomputed name no longer matches the file actually on disk, and the row silently disappears from this menu even though it's genuinely completed and archived. A proper fix would persist the archived filename at archive time instead of recomputing it.

New endpoint: `GET /api/invoices/completed`. New frontend page: `CompletedInvoices.jsx` (reuses `InvoiceResultsPanel` with `selectable` and `showStatus=False`). New menu key `completedinvoices` - added to `menuConfig.js`, `App.jsx`'s `PAGES/DEFAULT_ROLE_MENUS`, and `database.py`'s `_ROLE_MENU_DEFAULTS` (the seed only applies to a brand-new `tbl_RoleMenu`, so an existing deployment needs a Super Admin to grant it via Screen Access, or Super Admin/Developer see it automatically regardless).

User Management: login history

New `database.get_user_login_history(user_id)` reads `tbl_Audit_Event` WHERE `Entity='USER'` (already populated by every login/logout - see `record_login/record_logout`/the idle-session auto-logout). New endpoint: `GET /api/users/{id}/login-history?user_id=<caller>` - gated by `_require_developer()` server-side, not just hidden in the UI.

Dashboard: Duplicate column

Frontend-only. `database.list_batches()` already returned a generic per-status counts map (any status name that occurs); `Dashboard.jsx`'s `BATCH_STATUS_COLS` just never included `DUPLICATE` as one of the rendered columns. Fixed by adding it to that list - no backend change needed.

What's New (post-v2.3 session) - Technical Notes, continued

Batch status: Rejected By Accounts folded into the Loaded tier

- database._batch_status_and_lock() no longer gives Rejected its own label. It's folded into the LOADED tier for the stage-label branch: effective_loaded = counts.get("LOADED",0) + counts.get("REJECTED BY ACCOUNTS",0), then the usual "least advanced reached tier" pick runs as before (9 Posted + 1 Rejected -> LOADED; all-Rejected -> LOADED too, since nothing else reached a tier). Rationale: a rejected invoice already passed through Loaded and needs the exact same next step as a plain Loaded one.

- database._batch_is_cleared() was changed the same way: cleared = LOADED + POSTED + COMPLETED + REJECTED BY ACCOUNTS counts. This is the function list_batches() uses to compute blocked_by (an earlier, not-cleared batch blocks a later one's download) and the old "only one Downloaded/In Progress batch at a time" check in app.py's download_batch (which matches on the literal label string). Net effect: a batch with only a rejection in it no longer blocks any other batch's download - only genuine unstated work (invoices still at Ready To Load) does that now. If re-deriving either of these, keep them consistent with each other - the label and the clear/blocking computation must agree on what counts as "resolved enough."

Stale-unresolved auto-park window: 31 -> 90 days

- database.STALE_STATUS_EXPIRY_DAYS changed from 31 to 90. Single source of truth - both database.expire_stale_unresolved() (DB-status path) and processor.py's call to config_store.expire_stale_files("UNSUPPORTED", database.STALE_STATUS_EXPIRY_DAYS, "MANUALLY UPDATED") (filesystem-age path for Unsupported, which never gets a tracker row) read the same constant. No other hardcoded day count exists for this.

Completed Invoices: orphan rows for ALL_INVOICES files with no database row

- database.completed_invoices() now also returns a row for every file physically present in ALL_INVOICES that has no matching tracker/header row at all - e.g. its whole batch was removed by a direct SQL DELETE script (used for live/UAT cleanup throughout this project's support work), which only ever touches the database, never this archive folder. Implementation: matched = {archived_name(r) for r in rows} (the real, DB-backed rows already computed); orphan_names = sorted(present - matched); each becomes a row with every DB-derived field blank (invoice_no, vendor, batch, status, etc. - that data is genuinely gone with the deleted record) and a synthetic NEGATIVE header_id (real header ids are always positive), assigned by enumerate() over the sorted orphan names so it's stable across repeated calls as long as the file set doesn't change. The negative id exists purely so the frontend's per-row selection state and History-button gating never collides two different orphan rows onto the same key.

- config_store.find_pdf() gained a fallback: if a filename isn't found anywhere under the normal walk (which deliberately skips ALL_INVOICES, same as before), it now also checks os.path.join(base, ALL_INVOICES_DIR, filename) as a last resort. This is what lets an orphan row's View/Download still work off its only remaining copy, since there's no original file in a status folder to find for these. This fallback is additive only - every existing call site's behavior for a file that IS found elsewhere is unchanged.

- Frontend: InvoiceResultsPanel.jsx's History button is now gated on row.header_id > 0 (previously rendered for every row when selectable/showStatus allowed it) - an orphan has no audit trail to show, and a negative id would 404 if queried anyway. Details was already naturally hidden for orphans since invoice_type is blank (its existing gate is invoice_type === "PART").

Dashboard: Super Admin read-only batch export

- New stored procedure usp_FetchBatchAll mirrors usp_FetchBatch's dynamic-column mechanism exactly, but with no IsActive/IsExcluded/status filtering in its @idset - every header ever tied to the batch comes back, whatever its current stage. New database.fetch_batch_readonly(batch_name, sheet_cols) calls it and returns the same {sheet: {columns, rows}} shape fetch_batch() does, including the same display-correctness fixups (Consignment Note No. prefix strip, freight-line override, Reservation Entry Source Subtype) - but with NONE of fetch_batch()'s side effects: no _assign_document_numbers/_assign_entry_numbers call, no conn.commit(), no re-read-after-write. Every [No.]/[Entry No.] comes back exactly as currently stored (blank if the batch was never downloaded).

PIIPS - Developer Manual

- New endpoint GET /api/batches/export?batch=...&user_id=... in app.py, gated by `_require_developer` (Super Admin only) - verified this adds no write to any table: no `log_event()` call, no `mark_batch_downloaded()`, confirmed by grepping the whole code path for UPDATE/INSERT/DELETE/commit()/log_event and finding none. This is deliberately a SEPARATE action from the real batch download (/api/batches/download) - built for a Super Admin to pull a clean snapshot of a batch's current DB state to compare against an Excel someone says looks wrong, without risking another write while investigating.
- Frontend: Dashboard.jsx gained a new first column (before Batch Name) in the Batches table, rendered only when `isSuperAdmin`, calling the new `api.js exportBatchFile()/batchExportUrl()` pair (mirrors the existing `downloadBatchFile()/batchDownloadUrl()` pattern).

Folder/API/Database Configuration: change tracking and History

- New `database.get_config_history(kind=None)` queries `tbl_Audit_Event` WHERE `Entity='CONFIG'`, optionally filtered to one Action value (`FOLDER_CONFIG_CHANGED` / `API_CONFIG_CHANGED` / `DB_CONFIG_CHANGED`).
- All three save endpoints in app.py now call `log_event()` after a successful save: `update_config()` (Folder Path) and `save_api_config()` (Service First URL) each log old -> new as plain text; `save_db_config()` logs only '<server> / <database>' -> '<server> / <database> (<Windows|SQL> auth)' - username and password are NEVER written to the audit row, by design. `ConfigModel` and `ApiConfigModel` both gained a `user_id` field to carry who made the change (`DbConfigModel` already had one, used for its existing `_require_developer` gate on re-configuring an already-set connection).
- New endpoint GET /api/config/history?kind=...&user_id=..., gated by `_require_developer` (Super Admin only) - same enforcement pattern as the Template/User History endpoints, verified with a live 403 test against a non-Super-Admin token.
- Frontend: `Configuration.jsx`, `ApiConfiguration.jsx`, and `DatabaseConfig.jsx` each gained a History button (`isSuperAdmin` only) opening the same timeline Modal markup pattern already used by Template/Invoice/User History. `ApiConfiguration.jsx` did not previously accept a user prop at all - it does now (`App.jsx` already passes ``user`` to every page component generically via `<Active user={user} onRoleMenusSaved={refreshRoleMenus} />`, so no `App.jsx` change was needed). `DatabaseConfig.jsx`'s History button is additionally gated on `!onSaved`, since that component is reused for the pre-login bootstrap screen (no database yet, no logged-in user) where History makes no sense regardless of role.

User Management: Change Password panel removed

- The "assign a new password for another user" panel (and its form) is removed from `UserManagement.jsx` entirely, for every role. Removed along with it: the `assignPw/assignPwMsg/assignPwBusy` state, the `onAssignPw` handler, and the `assignableUsers` memo (all dead code once the panel using them was gone). `canAssignFor()` was KEPT - it's still used by the Users table itself (gating the per-row Reset-password button and the editable Type dropdown), unrelated to the panel that was removed. The per-row "Reset password" action (resets to the username, not an arbitrary password) is a separate, unaffected code path.